
Data Preprocessing Foundations

Tidy data, missing values, and transformation

Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Learning objectives and route through the class

By the end of this class, you should be able to

- connect a data-quality problem to a preparation step;
- define an observational unit and reshape a table accordingly;
- distinguish missingness assumptions and compare handling strategies; and
- calculate standardization and binning in R or Python.

Class 02 introduced summaries of location and spread. This class checks the meaning and arrangement of the values behind those summaries. We begin with a short recap, then organize a grades table, examine missing values, and transform numeric variables. Regression and multiple imputation appear as conceptual previews. Their model fitting and uncertainty calculations are not required for Lab 02.

1 Preparing for the integrated practical

Lab 02 begins in Class B and builds on Lab 01. Bring your approved dataset, its data dictionary, and your record of earlier decisions. Work in R or Python, locally or in an approved cloud environment. No installation of the other language is required. Keep assessed code, outputs, and reports private until the submission deadline and instructor release. Follow the syllabus portfolio workflow and the Lab 02 handout for submission details.

2 Recap, summarizing data

2.1 Mean and median after one replacement

Consider the hypothetical example from Class 02. Ten people have an average income of \$50,000. Replacing the highest earner with someone earning far more can make the mean arbitrarily large, while the fifth and sixth ordered incomes stay unchanged. The median therefore stays unchanged in that example.

The result depends on which observation is replaced. Table 1 uses four-person examples. In each replacement, the new income exceeds all four original incomes.

Table 1. Median after replacing the lowest or highest income

Four incomes	Original median	Lowest replaced	Highest replaced
50k, 50k, 50k, 50k	50k	50k	50k
40k, 40k, 60k, 60k	50k	60k	50k
30k, 40k, 60k, 60k	50k	60k	50k

For 40, 40, 60, 60, the median is $(40 + 60)/2 = 50$. Replacing a lowest value produces 40, 60, 60, large and median 60. Replacing a highest value produces 40, 40, 60, large and median 50. When every value is 50, either replacement preserves the median. Thus replacing a highest value is a sufficient condition in these examples, not the only way the median can remain unchanged. The central ordered values determine the result.

2.2 Location, spread, and skew

Mean, median, and mode describe location. Variance, standard deviation, IQR, and range describe spread. Report the statistic's meaning and units along with its value. The 1.5 IQR fences flag potential outliers for investigation; a flag does not establish a recording error.

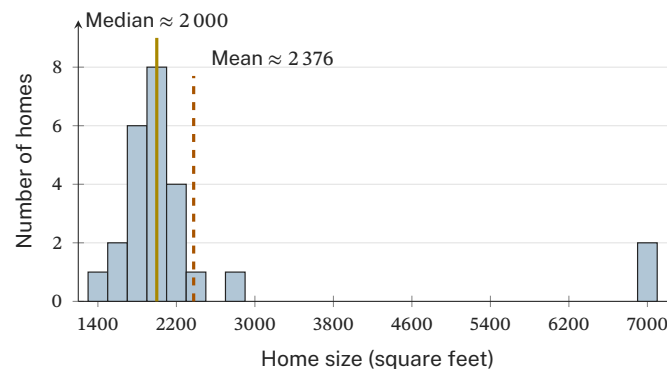


Figure 1. The shared Class 02 home-size teaching histogram

Figure 1 has 25 homes. Twenty-three fall between 1 400 and 2 800 square feet and two are near 7 000. Using bin midpoints gives an approximate median of 2 000 and mean of $59\,400/25 = 2\,376$. These are reconstructed teaching counts, not the unavailable original unbinned records. The distant homes pull the mean above the median. Which summary would you use for a typical home, and what information would it omit?

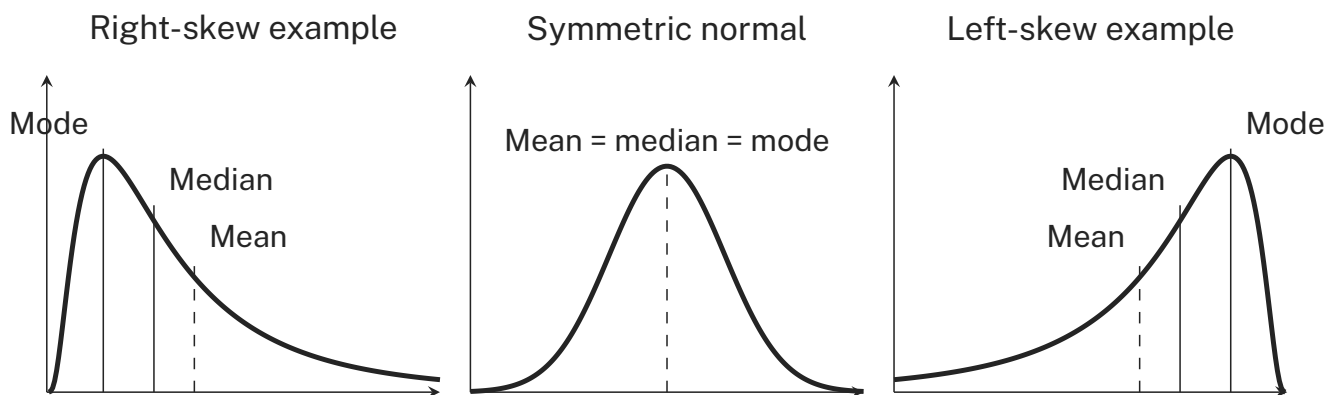


Figure 2. Computed examples of right skew, symmetry, and left skew

The ordering $\text{mode} < \text{median} < \text{mean}$ in the right-skew example is common but not a universal theorem. The reflected left-skew example reverses it. The normal example has equal mean, median, and mode.

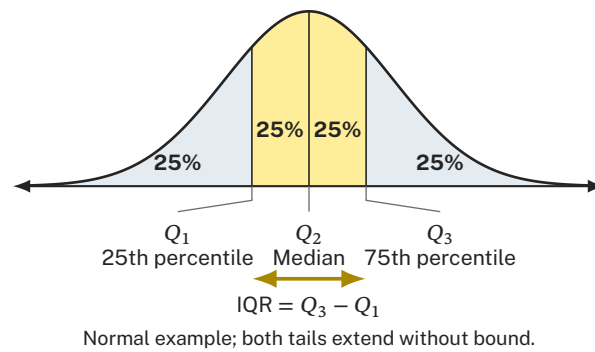


Figure 3. Quartiles divide the area of this continuous normal example into four equal parts

Figure 3 illustrates $IQR = Q_3 - Q_1$. The normal curve has unbounded tails, so its drawing has no finite minimum or maximum. For sample calculations, keep the interpolated quantile convention taught in Class 02.

3 Preparation in the course workflow

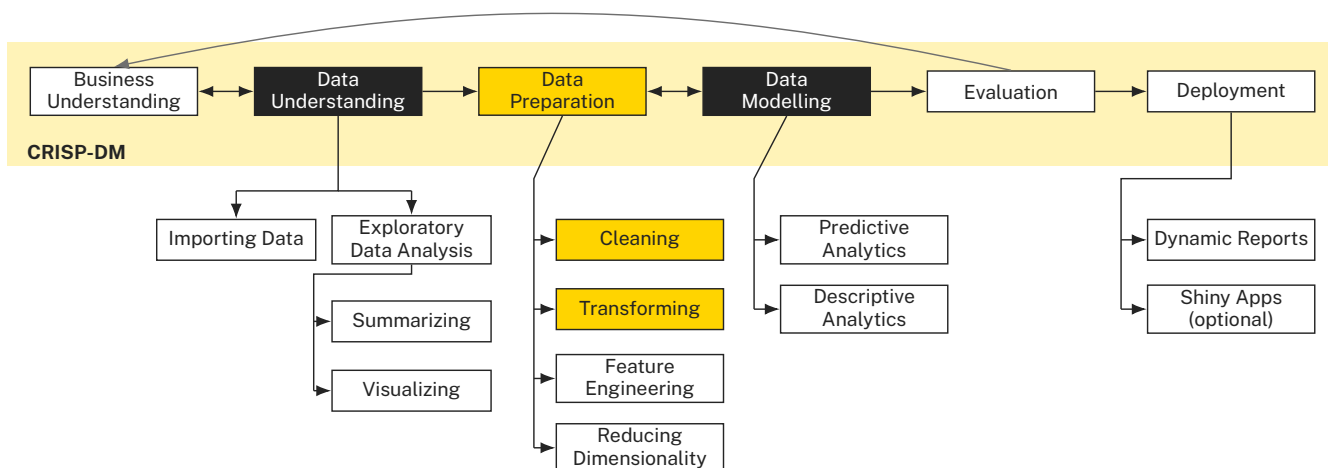


Figure 4. The Class 02 course map with Data Preparation highlighted

We move from data understanding into preparation while retaining the same workflow. Cleaning and transforming are today's focus. Later observations may send us back to check the original data; the process is iterative.

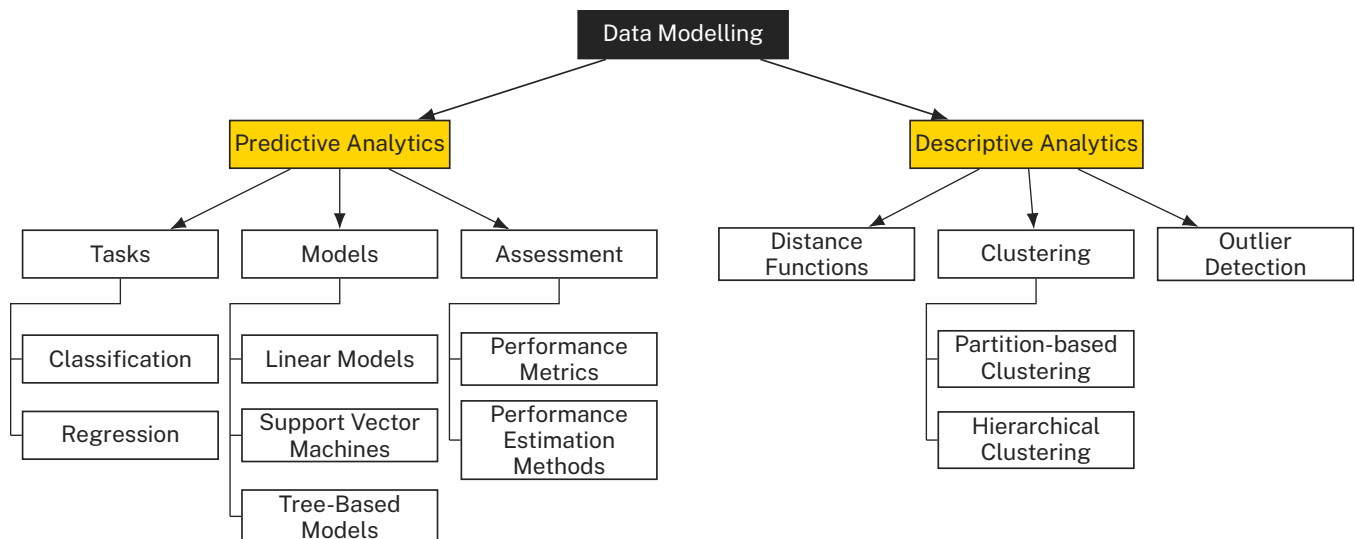


Figure 5. Later modelling topics, shown for orientation

Figure 5 is a map of later course material. Knowing how to fit or evaluate these models is not a pre-requisite for this class.

3.1 Motivations and categories

A mixed-unit temperature column calls for a unit conversion, for example $C = (F - 32) \times 5/9$. The value and unit change together. A documented missing code calls for a cleaning decision. A measured total and a count may support a new variable such as amount per observation. Too many variables can make analysis difficult and motivate dimensionality reduction.

Table 2. Problems and corresponding preparation steps

Problem	Preparation step
A temperature column mixes units	Transform to a common unit.
9999 is a documented missing code	Clean the representation of missingness.
A total and count support an amount-per-observation ratio	Create a derived variable.
Too many variables for a useful analysis	Consider dimensionality reduction.

Table 3. Four categories of data preparation

Cleaning	Interpret types, missing codes, inconsistencies, and errors.
Transformation	Change units, scale, or representation.
Feature engineering	Derive useful variables from recorded information.
Dimensionality reduction	Reduce the number of variables used in an analysis.

These categories can overlap. Reshaping may expose a cleaning problem, and a new variable may need a transformation. Class B extends today's cleaning and transformation with feature engineering and an introductory discussion of reducing the number of variables.

4 Data cleaning, tidy data

4.1 Define the observational unit first

Tidy structure

Use one variable per column, one observation per row, and one value per cell. Keep one type of observational unit in each table. Decide what counts as an observation before choosing the layout.

A student-level analysis can treat Math grade and English grade as two variables. For the next task, however, our observational unit is **one student in one course**. Then course name is a variable and grade is another variable. The same source information can need different layouts for different questions.

4.2 Diagnose the grades example

Table 4. Raw teaching data, with a documented missing-value code

Name	Math	English	DegreeYear
Luis	90	86	CS 2020
Maria	85	9999	Math 2013
Jose	70	60	CS 2015

All names and scores in Table 4 are invented. The example's data dictionary defines grades on a 0–100 scale and uses 9999 for a missing grade. It also defines DegreeYear as a degree label followed by an entrance year, separated by a literal pipe character.

Think for two minutes about the changes needed for one student-course row, then compare with a neighbour. There are three decisions.

1. Separate degree and entrance year into distinct variables.
2. Move course names into Course and scores into Grade.
3. Decode 9999 as missing using the data dictionary.

The last decision is a value-cleaning step, separate from the table reshape. An impossible value without documentation would require investigation first. Retain the raw data so that each transformation can be checked.

Table 5. One row per student and course after reshaping

Name	Course	Grade	Degree	Year
Luis	Math	90	CS	2020
Maria	Math	85	Math	2013
Jose	Math	70	CS	2015
Luis	English	86	CS	2020
Maria	English	NA	Math	2013
Jose	English	60	CS	2015

Three student rows become six student-course rows. Maria's missing English grade remains missing. No grade has been invented.

4.3 Runnable setup in either language

The following setup and examples can be run from top to bottom in a fresh session. The R version uses `tidyr`; the Python version uses `pandas` and `NumPy`. Choose either language. The tables, decisions, and expected results are the same. Keep grades as the raw input and transform its copy.

R

```
grades <- data.frame(  
  Name = c("Luis", "Maria", "Jose"),  
  Math = c(90, 85, 70),  
  English = c(86, 9999, 60),  
  DegreeYear = c("CS|2020", "Math|2013", "CS|2015")  
)  
# The example's data dictionary defines 9999 as missing.  
clean_grades <- grades  
clean_grades$English[clean_grades$English == 9999] <- NA  
#
```

Python

```
import numpy as np  
import pandas as pd  
  
grades = pd.DataFrame({  
    "Name": ["Luis", "Maria", "Jose"],  
    "Math": [90, 85, 70],  
    "English": [86, 9999, 60],  
    "DegreeYear": ["CS|2020", "Math|2013", "CS|2015"],  
})  
# The example's data dictionary defines 9999 as missing.  
clean_grades = grades.copy()  
clean_grades["English"] = clean_grades["English"].replace(9999, np.nan)  
#
```

4.4 Reshape the table

`separate_wider_delim` and `str.split` split the combined field. Here both interpret the pipe literally. Convert the year to an integer because it represents a year, not a category label. Then `pivot_longer` or `melt` moves the two course columns into a common grade column.

R

```
tidy_grades <- tidyr::separate_wider_delim(  
  clean_grades, DegreeYear, delim = "|",  
  names = c("Degree", "Year")  
)  
tidy_grades$Year <- as.integer(tidy_grades$Year)  
tidy_grades <- tidyr::pivot_longer(  
  tidy_grades, cols = c("Degree", "Year"),  
  values_from = English, names_to = "Course",  
  values_fill = NA
```

```
tidy_grades, c(Math, English),
names_to = "Course", values_to = "Grade",
cols_vary = "slowest"
)
tidy_grades <- tidy_grades[c("Name", "Course", "Grade", "Degree", "Year")]
print(tidy_grades)
#
```

Python

```
tidy_grades = clean_grades.copy()
tidy_grades[["Degree", "Year"]] = (
    tidy_grades["DegreeYear"].str.split("|", expand=True, regex=False)
)
tidy_grades["Year"] = tidy_grades["Year"].astype(int)
tidy_grades = tidy_grades.drop(columns="DegreeYear").melt(
    id_vars=["Name", "Degree", "Year"],
    var_name="Course", value_name="Grade",
)
tidy_grades = tidy_grades[["Name", "Course", "Grade", "Degree", "Year"]]
print(tidy_grades)
#
```

The output has six rows, five columns, and one missing grade. R's `cols_vary="slowest"` keeps all Math rows before all English rows, matching pandas `melt`. Both versions explicitly select the same column order. The older `tidyr` `gather` and `separate` functions remain in older examples; the code here uses the current replacements.

4.5 Dates and strings

Parsing a date requires its format. A timestamp also needs a time zone; a calendar date alone does not identify an instant. For example, a local timestamp must not be treated as UTC without checking how it was recorded. In R, `lubridate` provides date and time tools; Python provides `datetime`, `zoneinfo`, and pandas date/time operations.

Table 6. Date and string tools in R and Python

Task	R	Python equivalent
Handling dates and time zones	<code>lubridate</code>	<code>datetime</code> , <code>zoneinfo</code> , <code>pandas.to_datetime</code> with <code>tz_localize/tz_convert</code>
Handling strings	base R: <code>grep</code> , <code>strsplit</code> , <code>nchar</code> , <code>substr</code> , etc.; then <code>stringr</code>	<code>str</code> methods, the <code>re</code> module, the pandas <code>.str</code> accessor
More complex string cases	<code>stringi</code>	<code>regex</code> module, <code>unicodedata</code>

For labels, case and surrounding spaces may be accidental differences. Only normalize them if that change preserves meaning in the data dictionary.

R

```
labels <- c(" North ", "north", "SOUTH")
trimws(tolower(labels))
# "north" "north" "south"
as.Date(c("2026-09-15", "2026-09-16"),
        format = "%Y-%m-%d")
# Calendar dates; timestamps also need an explicit time zone.
#
```

Python

```
labels = pd.Series([" North ", "north", "SOUTH"])
print(labels.str.strip().str.lower())
# "north", "north", "south"
pd.to_datetime(["2026-09-15", "2026-09-16"], format="%Y-%m-%d")
# Calendar dates; timestamps also need an explicit time zone.
#
```

The normalized labels are north, north, and south. R offers base string tools, `stringr`, and `stringi`; Python offers string methods, regular expressions, and Unicode tools. Additional packages are options, not a requirement to switch language. The same cleaning decisions apply in either.

5 Data cleaning, missing values

5.1 Three mechanisms

Assumptions about the recording process

MCAR, missing completely at random. Missingness is independent of observed and unobserved values.

MAR, missing at random. After accounting for observed information, missingness does not depend on the unobserved values.

MNAR, missing not at random. Missingness still depends on unobserved information after accounting for what was observed.

These statements describe how gaps arise. They do not say that the values of a variable are themselves random, nor can a missingness percentage determine the mechanism.

Table 7. Hypothetical mechanisms and the assumptions that distinguish them

	Instrument example	Survey example
MCAR	Randomly selected readings are lost independently of measured values and recording order.	A random subset of forms is lost independently of responses.
MAR	Failures depend on recorded surface type; within each type they do not depend on the reading.	Response depends on recorded delivery method; within that method it does not depend on the answer.
MNAR	An unrecorded extreme reading makes failure more likely.	Response still depends on the undisclosed answer after accounting for recorded variables.

For the MAR examples, the conditioning information must be recorded. A battery failure is only a useful MCAR example when its occurrence is independent of the relevant measurements and recording order. Otherwise a plausible-sounding story can hide systematic missingness.

We can check whether gaps are associated with observed variables, which may challenge MCAR. The observed data alone cannot distinguish MAR from MNAR. That distinction needs knowledge of collection, additional information, or explicit assumptions. For a sensor that fails more outdoors, MAR would require that failure no longer depends on the unobserved reading after accounting for recorded location. Dependence on an unrecorded extreme reading would instead suggest MNAR.

5.2 Three broad responses

- **Omission** uses observed data and discards selected incomplete entries or rows.
- **Imputation** supplies estimated values for gaps, while retaining their missingness record.
- **Model-based handling** represents missing values within the analysis model instead of first making one complete table. This is a later-method preview, not a Lab 02 task.

None is a universal repair. Choose a method in relation to the question, the recording process, and the information available. Leaving a value missing is sometimes the most defensible decision.

5.3 Complete cases and available cases

Complete-case deletion uses rows observed on every variable needed for an analysis. Available-case summaries use every observed value needed for each summary. A two-variable statistic uses complete pairs, which is why the method is also called pairwise deletion.

Table 8. Complete-case and available-case trade-offs

Complete-case deletion	Available-case summaries
Every required variable is observed for each retained row.	Each summary uses the values observed for its own variables.
The summaries describe one common subset.	Different summaries may use different subsets and counts.
Can discard otherwise usable values.	Comparisons require care about which rows contributed.

If the original sampling process supports an unbiased mean, MCAR missingness preserves that property. Fewer observations still mean less information. MCAR cannot correct bias already present in

the sampling process. When missingness is systematic, the observed subset may differ from the population of interest.

In the grades example, requiring both scores keeps Luis and Jose. Their means are Math 80 and English 73. Using every available grade gives Math $245/3 \approx 81.67$ from three values and English $146/2 = 73$ from two values. The different counts must accompany the results.

R

```
scores <- clean_grades[c("Math", "English")]
complete <- scores[complete.cases(scores), ]
nrow(complete) # 2
colMeans(complete)
# Math 80, English 73
colMeans(scores, na.rm = TRUE)
# Math 81.66667, English 73
colSums(!is.na(scores))
# Math 3, English 2
#
```

Python

```
scores = clean_grades[["Math", "English"]]
complete = scores.dropna()
print(len(complete)) # 2
print(complete.mean())
# Math 80, English 73
print(scores.mean())
# Math 81.66667, English 73
print(scores.notna().sum())
# Math 3, English 2
#
```

R requires `na.rm=TRUE` for these column means; pandas skips missing values by default. Neither default determines whether omission is defensible.

Optional extension. Correlation, a measure of association between two numeric variables, is taught later. R's `cor` needs an explicit missing-value rule, while pandas correlation uses available pairs by default. If different pairs use different rows, a collection of correlations need not describe one common sample. Matrix methods impose further conditions; those details are outside this week's required work.

5.4 Imputation families

Table 9. Imputation strategies and the information each uses

Central value	A mean, median, or mode, selected for the variable type.
Ordered measurements	Previous value (LOCF), baseline value (BOCF), or interpolation between observations. Requires a justified time-series assumption.
Prediction	Other recorded variables inform a predicted numeric value or category. Conceptual preview.
Multiple imputation	Several plausible completed datasets and a combined analysis that represents uncertainty. Conceptual preview.

A central-value substitution might use a mean, median, or mode. Those choices are not interchangeable. The mean and median apply to numeric variables; the mode can apply to categorical variables. A mode substitution must state how ties between equally frequent categories are handled.

5.5 Compare mean fill with median fill

Extend the eight observations from Class 02 with two missing entries. Retain `with_gaps`; create separate filled copies. The mean rule fills with 10.5, and the median rule fills with 6.

R

```
observed <- c(2, 4, 4, 5, 7, 9, 12, 41)
with_gaps <- c(observed, NA, NA)
mean_filled <- with_gaps
mean_filled[is.na(mean_filled)] <- mean(observed)
median_filled <- with_gaps
median_filled[is.na(median_filled)] <- median(observed)
c(mean(mean_filled), sd(mean_filled)) # 10.5, 11.22497
c(mean(median_filled), sd(median_filled)) # 9.6, 11.3842
#
```

Python

```
observed = pd.Series([2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
with_gaps = pd.concat([observed, pd.Series([np.nan, np.nan])], ignore_index=True)
mean_filled = with_gaps.fillna(observed.mean())
median_filled = with_gaps.fillna(observed.median())
print(mean_filled.mean(), mean_filled.std()) # 10.5, 11.22497
print(median_filled.mean(), median_filled.std()) # 9.6, 11.3842
#
```

Mean fill preserves the observed mean of 10.5. Median fill changes the resulting mean to 9.6. The effects of one substitution rule must not be claimed for all central-value rules.

5.6 Calculate what mean fill does to spread

Predict the change in sample standard deviation before reading the calculation. The original observed mean is 10.5, and the squared deviations sum to 1134. Each mean-filled gap adds a zero deviation while increasing the sample count.

The same arithmetic as the slide example

Observed values ($n = 8$)	Two mean-filled gaps ($n = 10$)
$\bar{x} = 84/8 = 10.5$	$\bar{x} = 105/10 = 10.5$
$\sum_i (x_i - \bar{x})^2 = 1134$	$1134 + 0 + 0 = 1134$
$s_x^2 = 1134/7 = 162$	$s_x^2 = 1134/9 = 126$
$s_x = \sqrt{162} \approx 12.73$	$s_x = \sqrt{126} \approx 11.22$

The standard deviation falls from approximately 12.73 to 11.22, a reduction of about 11.8% relative to the observed-data SD. The true complete-data SD is unknown because the two original values are missing.

The observed mean stays exactly unchanged for *any* missingness mechanism. MCAR concerns whether that mean estimates the population mean without selection bias. It is a different statement from the arithmetic identity. Mean fill can distort relationships with other variables. Treating the filled values as if they were observed can also understate uncertainty; it does not restore the information that was lost.

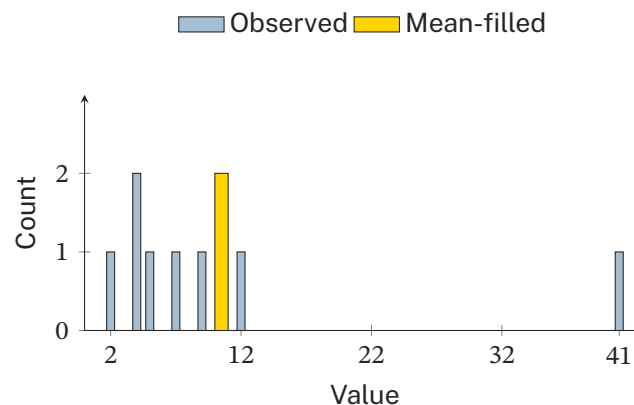


Figure 6. The eight observed values and two additional values imputed at the mean

5.7 Optional extension, ordered measurements

Last observation carried forward (LOCF) uses the most recent observed value. Baseline observation carried forward (BOCF) uses the first, baseline value. Backward fill uses a *later* observed value and is a different operation. Interpolation estimates values between observations; it is one way to impute interior gaps, not a replacement name for all imputation.

For one subject measured at five equally spaced, ordered times, consider 10, missing, 14, missing, 18. Forward fill gives 10, 10, 14, 14, 18; baseline fill gives 10, 10, 14, 10, 18; straight-line interpolation gives 10, 12, 14, 16, 18. None is justified merely because it is easy to compute. The examples below preserve all five positions. They contain no endpoint gaps.

R

```
# One subject, equally spaced and ordered times. No endpoint gaps.
series <- c(10, NA, 14, NA, 18)
last <- NA_real_
forward <- series
for (i in seq_along(series)) {
```

```

if (!is.na(series[i])) last <- series[i]
forward[i] <- last
}
baseline <- series
baseline[is.na(baseline)] <- series[1]
inside <- approx(seq_along(series), series,
                xout = seq_along(series))$y
forward          # 10, 10, 14, 14, 18
baseline         # 10, 10, 14, 10, 18
inside           # 10, 12, 14, 16, 18
#

```

Python

```

# One subject, equally spaced and ordered times. No endpoint gaps.
series = pd.Series([10, np.nan, 14, np.nan, 18])
forward = series.ffill()
baseline = series.fillna(series.iloc[0])
inside = series.interpolate(method="linear", limit_area="inside")
print(forward.tolist())      # 10, 10, 14, 14, 18
print(baseline.tolist())     # 10, 10, 14, 10, 18
print(inside.tolist())       # 10, 12, 14, 16, 18
#

```

Before applying these operations to real data, sort by time within each subject or series, check the spacing, and choose an endpoint policy. Do not carry values between different subjects or across an unjustified time gap. Baseline fill requires a known baseline. Interpolation needs observed values on both sides and a reasonable assumption about change between them.

6 Conceptual previews, prediction and uncertainty

6.1 Regression imputation

A regression model predicts a numeric variable from other recorded variables. A classification model predicts a category. Model fitting is taught later; for now, consider how replacing a gap by a prediction differs from replacing all gaps by the same mean.

A fitted prediction is an expected value under a model. Individual observed values usually vary around it. Filling every gap with the fitted prediction omits that residual variation and can make a relationship appear stronger than it is. Predictions can use useful information, but they do not reveal the unknown original values.

Table 10. What a regression-based imputation can and cannot provide

Potential benefit	Limitation
Uses other recorded information.	Needs a suitable model and observed predictors.
Predicts an expected numeric value.	Does not recover the unknown individual value.
Can improve on a single overall mean.	Deterministic predictions omit residual variation and can exaggerate associations.

Validity under MAR requires a suitable conditional model and appropriate observed predictors. Including one variable related to missingness is not a general guarantee of unbiased results. Near-perfect reconstruction is only plausible when a justified relationship nearly determines the missing value.

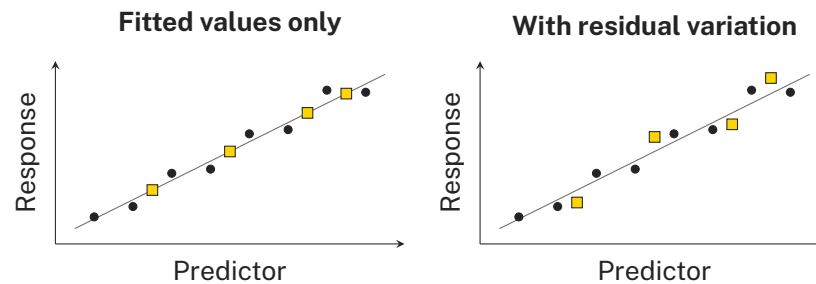


Figure 7. Fitted-value substitution and one illustrative stochastic alternative

In Figure 7, black circles are observed values, and the line is fitted to those circles. Gold squares identify filled values. The deterministic panel places them on the fitted line. The stochastic panel adds an illustrative residual deviation to each prediction. A suitable stochastic method represents residual variation; an arbitrary draw alone does not establish valid inference. Apparent relationships and their uncertainty must still be assessed.

6.2 Multiple imputation

Instead of retaining one set of guessed values, multiple imputation creates m plausible completed datasets. Each is analyzed separately, then the estimates and uncertainty are combined. Here m denotes the number of completed datasets, not a larger number of observed people or measurements.

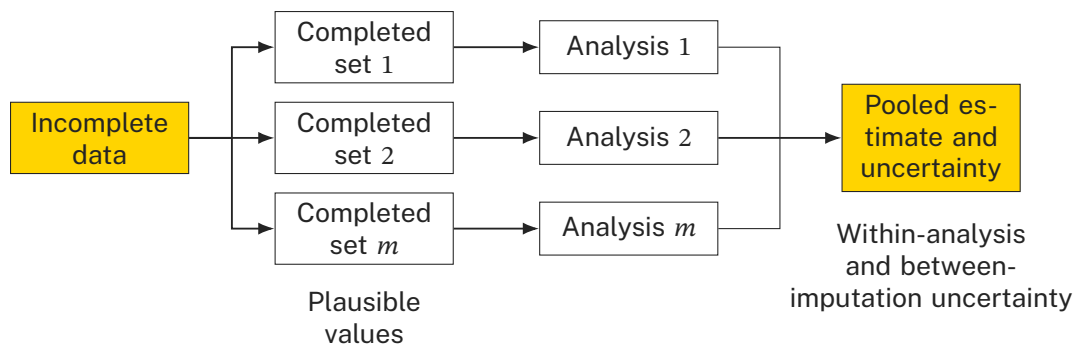


Figure 8. Create completed datasets, analyze each, and combine the results

The combined uncertainty includes both ordinary sampling uncertainty within an analysis and variation between the completed datasets. The spread between answers records uncertainty about the missing values. Standard applications assume MAR and suitable imputation and analysis models. MNAR requires additional assumptions and sensitivity analysis, which asks how conclusions change under different plausible missing-value scenarios. Small samples or high missingness do not automatically become reliable through repeated imputation. The pooling calculation is often called Rubin's rules; its implementation is covered in the optional reading.

6.3 Optional tools for later study

R's `recipes` organizes preparation steps, including separate rules for numeric and categorical predictors. Python's `SimpleImputer` provides a stated substitution rule; its default is the mean, but a median or most-frequent value can be selected. Apply a rule compatible with each variable's type.

Nearest-neighbour imputation borrows values from similar records. Its distance rule matters. The R and Python implementations need not define similarity the same way, and the units of a numeric variable can affect that comparison. Selecting and checking a distance is later course work.

The R `mice` package and Statsmodels MICE in Python support repeated imputation and analysis. Pooling combines the fitted-model estimates, not the rows of the completed datasets. R's default imputation method depends on the variable type; predictive mean matching for numeric data draws an observed donor value with a similar predicted value. That is one method, not a universal rule for categorical data. A single result from Python's `IterativeImputer` is not automatically multiple-imputation analysis. The optional reading explains the workflows; implementing them is not required this week.

6.4 Choose a strategy at the current level

Given two missing measurements, first state the question, the available information, and a plausible explanation for the gaps. Report observed values and counts when that supports the question. If filling a gap is defensible, name the rule, retain the missingness record, and explain its limitation. Seek additional information when the decision is unsupported. More elaborate software is not a substitute for that explanation.

7 Data transformation

7.1 Standardizing numeric variables

For at least two observed numeric values with positive sample SD, define

$$z_i = \frac{x_i - \bar{x}}{s_x}.$$

The result expresses distance from the sample mean in sample-SD units. For $x = 2, 4, 4, 5, 7, 9, 12, 41$, $\bar{x} = 10.5$ and $s_x = \sqrt{162} \approx 12.73$. Thus $z_1 \approx -0.67$, $z_4 \approx -0.43$, and $z_8 \approx 2.40$.

The transformed values have mean 0 and sample SD 1, apart from numerical rounding. Their shape and ordering remain the same. The potential outlier is still unusual; changing units does not repair a data problem. Constant columns have zero SD and require a separate documented decision instead of division by zero. Missing values remain missing unless an earlier justified step handles them.

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
stopifnot(length(x) > 1, sd(x) > 0)
z <- (x - mean(x)) / sd(x)
z[1] # -0.6678231
mean(z) # approximately 0
sd(z) # 1
as.vector(scale(x))[1] # -0.6678231
#
```

Python

```
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
assert len(x) > 1 and x.std(ddof=1) > 0
z = (x - x.mean()) / x.std(ddof=1)
print(z.iloc[0]) # -0.6678231
print(z.mean()) # approximately 0
print(z.std(ddof=1)) # 1
# Both examples use the sample SD.
#
```

Both examples use divisor $n - 1$ for sample SD, matching Class 02. R's default `scale(x)` produces the same centered result. NumPy's default SD and scikit-learn's `StandardScaler` use divisor n . For these eight values that gives a first standardized value of approximately -0.713933 , rather than -0.667823 . State the convention used.

Optional modelling connection. Methods that compare numerical distances can be affected by units, for example when a dollar-valued feature has a much larger range than an age in years. Later modelling classes develop this issue for nearest neighbours and related methods. When evaluating a predictive model, preparation parameters must be learned from its training data and applied to separate evaluation data. Summarizing or transforming a descriptive dataset does not by itself create that train/test setting. We will introduce the full evaluation workflow before requiring it in assessed work.

7.2 Discretizing numeric variables

Discretization replaces numeric values with categories. Some methods require categorical input, and a small number of categories can simplify communication or computation. It also loses numerical detail. Choose boundaries for the question rather than treating any particular binning as automatically correct.

Equal-width bins divide a chosen numeric range into k intervals of equal width. Quantile-based bins aim for similar counts. Ties can prevent equal counts because equal values should not be split across categories merely to balance them. Here k is the number of bins and n remains the number of observations.

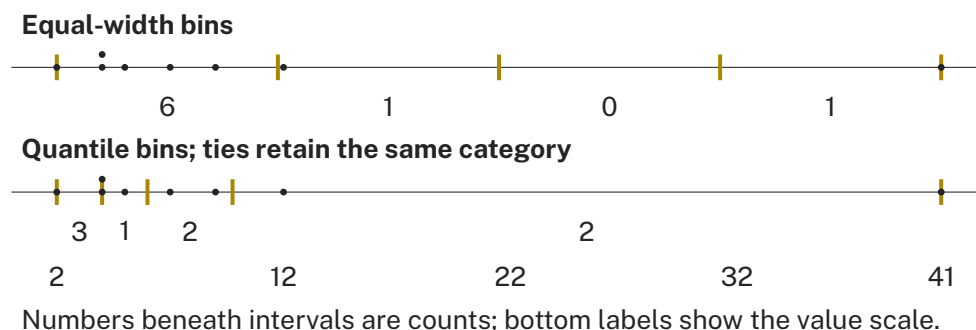


Figure 9. The same eight observations under two four-bin rules

For the running vector, equal-width counts are 6,1,0,1. Quantile-bin counts are 3,1,2,2 because the boundary at 4 includes both observations equal to 4. The two marks at that x-coordinate are stacked vertically in Figure 9; neither value is shifted numerically. The distant observation at 41 leaves several

equal-width bins sparse. That may be useful for displaying the tail, but unhelpful if the purpose is to form similarly sized groups.

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
k <- 4
width_edges <- seq(min(x), max(x),
                   length.out = k + 1)
quantile_edges <- quantile(
  x, probs = seq(0, 1, 1/k))
table(cut(x, width_edges,
          include.lowest = TRUE))
# 6, 1, 0, 1
table(cut(x, quantile_edges,
          include.lowest = TRUE))
# 3, 1, 2, 2
#
```

Python

```
x = pd.Series([2, 4, 4, 5, 7, 9, 12, 41])
k = 4
width_edges = np.linspace(
    x.min(), x.max(), k + 1)
quantile_edges = x.quantile(
    np.linspace(0, 1, k + 1))
print(pd.cut(x, width_edges,
             include_lowest=True).value_counts(sort=False))
# 6, 1, 0, 1
print(pd.cut(x, quantile_edges,
             include_lowest=True).value_counts(sort=False))
# 3, 1, 2, 2
#
```

Both implementations explicitly include the minimum. Save and reuse the boundaries when categories must be comparable across datasets. Re-estimating minimum, maximum, or quantiles separately can change category meanings. If quantile boundaries repeat, reduce the number of bins and document that choice. Do not silently move tied observations.

Convenience functions can compute boundaries automatically. R's `cut(x, breaks=k)` and pandas `cut(x, k)` slightly extend the range to include its endpoints; their printed labels can differ in rounding. With explicit boundaries, the examples above state the minimum-inclusion rule. pandas `qcut` creates quantile bins directly and raises an error for non-unique boundaries unless told to drop duplicate edges. Saved edges provide reuse without introducing a modelling pipeline.

8 Closing think-pair-share, age non-disclosure

A hypothetical survey records age and consumption. Respondents under 21 are less likely to disclose age even at the same consumption level. No other variables are recorded. What does this suggest about missingness, and could age categories recover ages that were never reported?

Under the stated setup, age-dependent missingness suggests MNAR relative to the recorded information. Additional observations or a different collection process could alter that assessment. The simple methods considered here do not provide a general guarantee of removing this selection bias.

Age bins can summarize observed ages, but a refusal does not establish that the respondent is under 21. Keep unknown ages missing. A separate nominal “age not disclosed” flag records the missingness event without inventing an age. It can help describe the recording process, but the flag does not itself resolve MNAR bias or recover the unobserved ages. Do not merge “under 21” and “not disclosed” into a single age level.

9 Summary and Lab 02 connection

Define the observational unit before reshaping. Use the data dictionary to interpret units, types, and missing codes. Report counts with summaries and justify deletion or imputation. Standardization changes scale; binning reduces detail. Keep the raw data and record transformations so another person can follow the reasoning.

Lab 02 extends Lab 01 with inspection, cleaning, documented decisions, and before/after summaries. Follow its handout for the required operations; it does not require imputation models, PCA, or other model fitting. Work privately, commit corrections as new commits, and preserve the submitted snapshot and its history. Public sharing begins only after the deadline and instructor release.

10 Next lecture

Class B continues data preparation, adds derived variables, and introduces reducing the number of variables. The practical develops the steps already explained before asking you to apply them.

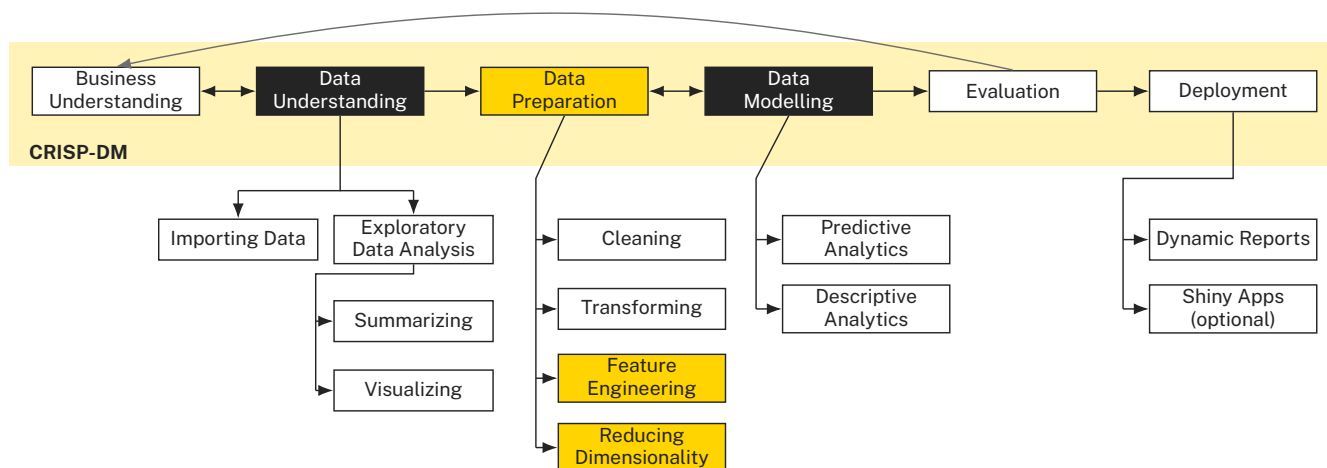


Figure 10. Continue the same preparation stage in Class B

Readings and optional extensions

- Wickham, *Tidy Data* (2014). Observations, variables, and table structure.
- `tidyr`, `pivot_longer` and `pandas`, `melt`. Read the examples for your chosen language.
- van Buuren, *Flexible Imputation of Missing Data*, Section 1.2. Missingness mechanisms.

- Optional extension, Section 1.3 on imputation trade-offs and the mice pooling workflow. Model fitting and pooling are not required for Lab 02.